

Localisation

Definizione generale.

La *localizzazione* è la capacità di un programma di adattarsi al luogo geografico e culturale in cui viene eseguito. Non significa solo tradurre le stringhe di testo, ma anche gestire correttamente:

- l'ortografia delle parole (es. "localisation" in inglese britannico vs "localization" in inglese americano),
- il formato dei numeri (dove va la virgola, dove va il punto),
- il formato delle valute (€, \$, £),
- il formato delle date e degli orari.

Un dettaglio curioso che il docente usa come esempio: lui scrive "localisation" con la **S** (inglese irlandese/britannico), mentre Oracle, azienda americana, scrive "localization" con la **Z**. Questa piccola differenza è già di per sé un caso di localizzazione.

Introduction and Creating Locales

Definizione di Locale. Un **Locale** è, detto in modo semplice, un'accoppiata **lingua + paese**.

Rappresenta chi è l'utente: che lingua parla e in che nazione si trova. La classe si trova nel pacchetto `java.util`.

La regola di scrittura del Locale (fondamentale).

- La **lingua** si scrive in **minuscolo** (es. `en` per English).
- Il **paese** si scrive in **MAIUSCOLO** (es. `IE` per Irlanda).
- Il separatore è l'**underscore** `_`.
- Formato: `lingua_PAESE` → esempio `en_IE` = inglese in Irlanda.
- Il paese è **facoltativo**; la lingua no.

Il docente usa un trucco mnemonico per ricordare l'ordine (lingua minuscola prima, paese maiuscolo dopo): pensa a "**L**eaving **C**ert" (l'esame di stato irlandese). Se scrivi l'ordine sbagliato o le maiuscole/minuscole sbagliate, il locale non funziona correttamente.

Come ottenere il locale predefinito del sistema:

```
Locale.getDefault(); // metodo statico
```

Sul portatile del docente questo restituisce `en_IE` → la macchina usa inglese ed è collocata in Irlanda.

I tre modi per creare un Locale diverso da quello predefinito:

a) Con il costruttore. Passi solo la lingua, oppure lingua + paese.

```
Locale l1 = new Locale("en"); // solo inglese
```

```
Locale l2 = new Locale("en", "GB"); // inglese in Gran Bretagna
```

```
l2.getDisplayLanguage(); // "English"  
l2.getDisplayCountry(); // "United Kingdom"
```

Nota: Gran Bretagna (GB) e Regno Unito (UK) qui coincidono.

b) Con le costanti già pronte. La classe `Locale` fornisce costanti per i locale più comuni.

```
Locale fr = Locale.FRANCE; // francese in Francia (fr_FR)  
fr.getDisplayLanguage(); // "French"  
fr.getDisplayCountry(); // "France"
```

c) Con il Builder (design pattern Builder). È il più flessibile perché puoi specificare le proprietà in qualsiasi ordine.

```
Locale ar = new Locale.Builder()  
    .setLanguage("ar") // arabo  
    .setRegion("EG") // Egitto  
    .build();  
// risultato: arabo in Egitto (ar_EG)
```

// L'ordine non conta:

```
Locale ar2 = new Locale.Builder()  
    .setRegion("KW") // metto prima il paese (Kuwait)  
    .setLanguage("ar") // poi la lingua (arabo)  
    .build();  
// risultato: arabo in Kuwait (ar_KW)
```

Il punto chiave del Builder che il docente sottolinea: puoi mettere prima il paese e poi la lingua, o viceversa — il risultato è corretto lo stesso.

Localising Numbers and Currencies – introduzione

Definizione. Sia i numeri che le valute si scrivono in modo diverso da un paese all'altro. Java usa la classe astratta **NumberFormat** (pacchetto `java.text`) per gestire queste conversioni.

I metodi statici di fabbrica (factory methods) di `NumberFormat` restituiscono un oggetto formattatore. I principali:

- `getInstance()` / `getNumberInstance()` → per i numeri normali,
- `getCurrencyInstance()` → per le valute,
- `getPercentInstance()` → per le percentuali.

Ognuno esiste in **due versioni** (overload): una senza argomenti (usa il locale predefinito) e una a cui passi un `Locale` specifico.

Le due direzioni di conversione. Una volta ottenuto il formattatore, hai due operazioni opposte:

- `format(numero)` → converte un **numero in una stringa** (numero → testo leggibile),
- `parse(stringa)` → converte una **stringa in un numero** (testo → numero).

Localising Numbers and Currencies (esempio pratico)

Il metodo `format` sui numeri. Locale predefinito del docente: `en_IE`.

```
double n = 77000.11;
```

```
NumberFormat.getInstance().format(n);           // predefinito (en_IE) → 77,000.11
```

```
NumberFormat.getNumberInstance(Locale.US).format(n); // USA → 77,000.11
```

```
NumberFormat.getNumberInstance(Locale.ITALY).format(n); // Italia → 77.000,11
```

```
NumberFormat.getNumberInstance(Locale.FRANCE).format(n); // Francia → 77 000,11
```

Cosa cambia da paese a paese:

- **Irlanda/USA:** virgola `,` per le migliaia, punto `.` per i decimali → `77,000.11`
- **Italia:** l'opposto → punto per le migliaia, virgola per i decimali → `77.000,11`
- **Francia:** **spazio** come separatore delle migliaia → `77 000,11`

Il metodo `format` sulle valute. Numero di esempio: `23.22`.

```
double c = 23.22;
```

```
NumberFormat.getCurrencyInstance().format(c);           // en_IE → €23.22
```

```
NumberFormat.getCurrencyInstance(Locale.US).format(c);  // USA → $23.22
```

```
NumberFormat.getCurrencyInstance(Locale.ITALY).format(c); // Italia → 23,22 €
```

```
NumberFormat.getCurrencyInstance(Locale.UK).format(c);  // UK → £23.22
```

Nota culturale importante: in Italia il simbolo dell'euro va **alla fine** (`23,22 €`), mentre in Irlanda/USA il simbolo va **prima** (`€23.22`).

Il metodo `parse` (direzione inversa: stringa → numero).

Attenzione: `parse` lancia un'eccezione controllata, `ParseException`, quindi va gestita.

```
NumberFormat nf = NumberFormat.getNumberInstance(Locale.US);
```

```
Number num = nf.parse("77,000.11"); // legge la stringa USA e restituisce 77000.11
```

Il concetto chiave che il docente ripete: **il locale che usi per fare il `parse` deve corrispondere al formato della stringa di input**. Se la stringa è nel formato italiano (`77.000,11`) devi usare

Locale . ITALY per interpretarla correttamente, altrimenti sbaglia il numero. Nell'esempio prende gli output prodotti prima con `format` e li ri-analizza con `parse` per tornare al numero originale — funziona per tutti e tre i locale (USA, Italia, Francia) proprio perché usa il locale giusto per ciascuna stringa.

Localising Numbers using custom formats

Definizione. Con la classe `DecimalFormat` (pacchetto `java.text`) puoi definire un formato **personalizzato** tramite un "pattern" (schema) fatto di caratteri speciali.

I due simboli chiave del pattern:

- `#` (hash/cancelletto) → "cifra opzionale": se in quella posizione non c'è una cifra, **non stampare niente** (la ometti).
- `0` (zero) → "cifra obbligatoria": se in quella posizione non c'è una cifra, **stampa uno zero**.

Esempio con #:

```
double d = 77000.171;
```

```
DecimalFormat df = new DecimalFormat("€###,###.#");
```

```
df.format(d); // → €77,000.2
```

Cosa succede qui:

- Il pattern prevede fino a 7 cifre prima della virgola (`###, ###`) ma il numero ne ha solo 5 → le posizioni mancanti vengono **omesse** (non riempite di zeri, grazie a `#`).
- Dopo il decimale il pattern prevede **una** sola cifra (`.#`), ma il numero ne ha due (`.171`) → viene **arrotondato** → `.2`.

Esempio con 0:

```
DecimalFormat df = new DecimalFormat("€0,000,000.00");
```

```
df.format(d); // riempie di zeri le posizioni mancanti
```

Con lo `0` invece dell'`#`, dove mancano le cifre vengono **inseriti degli zeri** anziché omettere.

L'arrotondamento sui decimali avviene comunque per eccesso.

Regola da ricordare per l'esame: `#` = ometti se manca, `0` = metti zero se manca. In entrambi i casi i decimali in eccesso vengono arrotondati.

Localising Dates

Definizione. Anche le date e gli orari cambiano formato da paese a paese. Java usa la classe `DateTimeFormatter` (pacchetto `java.time.format`), che offre metodi di fabbrica per costruire formati localizzati.

I metodi di fabbrica principali:

- `ofLocalizedDate(style)` → solo la data,
- `ofLocalizedTime(style)` → solo l'ora,
- `ofLocalizedDateTime(style)` → data + ora.

Gli stili disponibili (enum `FormatStyle`): `SHORT`, `MEDIUM`, `LONG`, `FULL`. Il docente precisa che per l'esame **non serve conoscere le differenze esatte** tra questi stili, basta sapere che esistono.

Per applicare un locale specifico si aggiunge il metodo `.withLocale(locale)`.

```
Locale us = Locale.US;
```

```
Locale fr = Locale.FRANCE;
```

```
Locale de = Locale.GERMANY;
```

```
LocalDateTime dt = LocalDateTime.now();
```

```
// esempio: 2024-...T13:25:11.<nanosecondi> → formato ISO completo (anno-mese-giorno)
```

```
// DATA in stile MEDIUM, locale USA:
```

```
DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM)
```

```
    .withLocale(us)
```

```
    .format(dt);
```

```
// stesso ma locale francese → il formato cambia leggermente:
```

```
DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM)
```

```
    .withLocale(fr)
```

```
    .format(dt);
```

```
// ORA in stile SHORT, locale USA:
```

```
DateTimeFormatter.ofLocalizedTime(FormatStyle.SHORT)
```

```
    .withLocale(us)
```

```
    .format(dt); // usa orologio a 12 ore con PM → es. 1:25 PM
```

```
// ORA in stile SHORT, locale tedesco:
```

```
DateTimeFormatter.ofLocalizedTime(FormatStyle.SHORT)
```

```
    .withLocale(de)
```

```
    .format(dt); // usa orologio a 24 ore → 13:49
```

Osservazioni chiave dall'esempio:

- **USA:** orologio a **12 ore** con AM/PM (es. 1 : 49 PM).
 - **Germania:** orologio a **24 ore** (es. 13 : 49).
 - **Ordine giorno/mese:** in Irlanda (predefinito) esce prima il **giorno** poi il mese; negli USA esce prima il **mese** poi il giorno.
 - Se **non** usi `.withLocale( . . . )`, viene usato il locale predefinito (nel suo caso `en_IE`).
-

Locale.Category enums

Definizione e problema. Quando imposti il locale predefinito con `Locale.setDefault( . . . )`, Java cambia **automaticamente** sia la parte di **visualizzazione** (i nomi di lingue e paesi) sia la parte di **formattazione** (numeri, valute, date). Ma a volte vuoi controllarle **separatamente**.

Le due categorie dell'enum `Locale.Category`:

- **DISPLAY** → riguarda la *visualizzazione* delle informazioni (i nomi: come viene scritto "English", "Irlanda", ecc.).
- **FORMAT** → riguarda la *formattazione* di numeri, valute e date.

Puoi impostarle una alla volta con:

```
Locale.setDefault(Locale.Category.DISPLAY, locale);
```

```
Locale.setDefault(Locale.Category.FORMAT, locale);
```

Esempio pratico. Locale predefinito di partenza: `en_IE`. Numero `n = 77000.11`.

```
Locale fr = Locale.FRANCE;
```

```
// Stato iniziale (tutto en_IE):
```

```
Locale.getDefault().getDisplayLanguage(); // "English"
```

```
Locale.getDefault().getDisplayCountry(); // "Ireland"
```

```
NumberFormat.getInstance().format(n);    // 77,000.11 (formato irlandese)
```

```
// Cambio SOLO la visualizzazione in francese:
```

```
Locale.setDefault(Locale.Category.DISPLAY, fr);
```

```
Locale.getDefault().getDisplayLanguage(); // ora in francese: "anglais"
```

```
Locale.getDefault().getDisplayCountry(); // "Irlande"
```

```
// MA il numero resta nel formato inglese/irlandese:
```

```
NumberFormat.getInstance().format(n);    // ancora 77,000.11
```

```
// Ora cambio ANCHE il formato in francese:
```

```
Locale.setDefault(Locale.Category.FORMAT, fr);
```

```
NumberFormat.getInstance().format(n); // 77 000,11 (formato francese)
```

Il concetto chiave da capire: cambiare DISPLAY non tocca FORMAT e viceversa. Nell'esempio, dopo aver cambiato solo DISPLAY, i **nomi** appaiono in francese ("anglais", "Irlande") ma i **numeri** restano ancora formattati all'inglese. Solo quando cambi anche FORMAT il numero passa al formato francese (spazio per le migliaia, virgola per i decimali).

Resource Bundles, Property Files and Searching

Definizione di Resource Bundle. Un **Resource Bundle** ("fascio di risorse") contiene i dati specifici del locale che il programma userà, tipicamente le stringhe tradotte. Ha la struttura di una **mappa (chiave → valore)**, dove i valori sono specifici per ogni locale. La classe è `ResourceBundle` (pacchetto `java.util`).

Due modi per creare un resource bundle:

1. **File Java** che estende la classe `ResourceBundle`.
2. **File di proprietà (.properties):** file di testo con coppie chiave=valore.

Il corso si concentra sui **property files**.

I Property Files.

- Sono semplici file di testo con coppie nel formato chiave=valore oppure chiave:valore.
- Sia la chiave che il valore sono **stringhe**.
- Locale diversi = file di proprietà diversi, distinti tramite convenzioni di denominazione.
- Il suffisso è **sempre** `.properties`.

Come caricare un bundle (metodo statico di fabbrica `getBundle` della classe `ResourceBundle`):

```
// solo nome del bundle:
```

```
ResourceBundle rb = ResourceBundle.getBundle("Team");
```

```
// nome + locale specifico:
```

```
ResourceBundle rb = ResourceBundle.getBundle("Team", Locale.GERMANY);
```

L'ordine di ricerca (search order) — concetto centrale. Quando chiedi un bundle, Java cerca i file in un ordine molto preciso, dal **più specifico al più generico**. Esempio: bundle chiamato `Team`, locale richiesto `de_DE` (tedesco in Germania), locale predefinito `en_IE`.

L'ordine di ricerca dei nomi file è:

1. `Team_de_DE.properties` → nome bundle + locale esatto richiesto (lingua+paese).

2. `Team_de.properties` → tolgo il paese, resta solo la lingua richiesta.
3. `Team_en_IE.properties` → passo al **locale predefinito** completo (lingua+paese).
4. `Team_en.properties` → tolgo il paese dal locale predefinito, resta la lingua.
5. `Team.properties` → il bundle **di default**, senza alcun locale.

Se alla fine nessuno esiste, viene lanciata un'eccezione (`MissingResourceException`).

Resource Bundle Searching – Example

Questa lezione mostra due meccanismi combinati con esempi concreti.

Meccanismo 1: la gerarchia si "blocca" (locks in) al primo file trovato.

Java prima cerca il file scendendo la gerarchia (come sopra). **Appena trova il primo file esistente**, quella gerarchia viene **bloccata**: da lì in poi, per cercare le singole **chiavi**, Java risale solo lungo quella catena di file, ignorando i livelli che stavano prima del file trovato.

Meccanismo 2: file `.properties` e file `.class` Java NON condividono la gerarchia. Per l'esame basta conoscere i property files. (In generale i file `.class` Java avrebbero priorità sui `.properties` allo stesso livello, ma qui si lavora solo con proprietà.)

Esempio A. File di proprietà disponibili sul disco: `en_CA` (inglese Canada), `en_IE` (inglese Irlanda), `en` (inglese), e il default. Locale predefinito: `en_IE`. Locale cercato: `en_CA`.

Ricerca (ricorda: prima cerca `.java/.class`, che non esistono, poi `.properties`):

1. `Team_en_CA.properties` → **trovato!** → la gerarchia si blocca qui.

Ora, bloccati su `en_CA`, per cercare una chiave (es. `name`) Java risale così:

1. `Team_en_CA.properties` → se non c'è la chiave...
2. `Team_en.properties` → tolto il paese...
3. `Team.properties` → il default.

Punto importante: anche se il locale predefinito è `en_IE`, il file `Team_en_IE.properties` **non viene mai consultato**, perché la gerarchia si era bloccata su `en_CA`, e `en_IE` non fa parte della catena "genitore" di `en_CA`.

Esempio B. Locale cercato: `en_US` (inglese USA). Non esistono file per `en_US`.

1. `Team_en_US.properties` → non esiste.
2. `Team_en.properties` → **trovato!** → gerarchia bloccata su `en`.

Ora il genitore di `en` è solo `Team.properties` (il default). Per cercare una chiave: prima in `Team_en.properties`, poi in `Team.properties`.

Esempio C (forzando l'uso del locale predefinito). Locale predefinito impostato a `en_IE`, locale cercato: `fr_FR` (francese in Francia). Non esistono file francesi.

1. `Team_fr_FR.properties` → non esiste.
2. `Team_fr.properties` → non esiste.
3. → **entra in gioco il locale predefinito `en_IE`**:

4. `Team_en_IE.properties` → **trovato!** → gerarchia bloccata su `en_IE`.

Da qui, per cercare le chiavi si risale: `Team_en_IE.properties` → `Team_en.properties` → `Team.properties`.

La lezione riassunta: il programma dimostra *come* vengono cercati i bundle (dal più specifico al più generico, passando dal locale richiesto a quello predefinito al default), e *come*, una volta trovato il primo file, la gerarchia di ricerca delle chiavi resti fissata su quella catena.